Appendix D
Benchmarking three actor substrates

Performance claims require a more precise subject than a language name. The actor interface described in this book has three implementations of immediate interest: browser actors implemented as SWI-WASM Web Workers, native actors implemented as SWI-Prolog threads, and Erlang processes running on the BEAM virtual machine. They expose similar message-passing patterns, but they realise an actor at very different levels of granularity.

This appendix compares those three substrates. It is not a contest between Prolog and Erlang as languages, nor does it attempt to reduce the usefulness of a programming model to a single number. Its purpose is narrower: to measure some costs that a programmer can observe, relate them to the implementations described in Appendix C, and identify where optimisation work would have the greatest effect.

D.1 What is being compared

Table D.1 summarises the three implementations. The unit called an actor is intentionally the same at the interface and intentionally different underneath.

Table D.1 The three actor substrates.

Substrate	Actor representation	Communication path
Browser node	An independent SWI-WASM virtual machine in a Web Worker	Messages relayed between Workers by the browser-side actor bridge
Native node	An SWI-Prolog thread with a private actor context	In-process delivery through the native actor runtime
Erlang	A lightweight BEAM process	In-VM delivery through an Erlang process mailbox

These differences make the result partly predictable. A BEAM process is designed to be created cheaply and in great numbers. A native Web Prolog actor is heavier, since it is built from a native Prolog thread and its execution context. A browser actor is heavier again: creating one means starting another Worker, instantiating SWI-WASM, installing its source, and connecting it to the page-level message router. The measurements tell us the size of these differences in the current implementations; they do not establish an intrinsic cost for Web Prolog.

D.2 Method

The measurements reported here were made on 29 July 2026 on a 2023 Apple iMac with an eight-core M3 processor and 16 GB of memory, running macOS 26.5.1. The native implementation used SWI-Prolog 10.1.3. The browser implementation used SWI-WASM 10.1.8 in the Worker-based browser node. The Erlang implementation used Erlang/OTP 26 with ERTS 14.2.5.

Three microbenchmarks were used:

Actor lifecycle.
: Start one actor, send it an instruction to stop, wait for its monitor notification, and repeat. The timed interval therefore includes actor creation, initialisation, source installation where required, one message, termination, and monitor delivery. It is an end-to-end actor-lifecycle measurement, not a measurement of the underlying thread or process creation primitive alone.

Chained spawning.
: Have each actor create one unlinked child and send it a message, producing a chain of actor creations. This measures the layer-0 native actor core separately from the full isolated node.

Steady-state ping-pong.
: Create two actors before timing begins, warm them up with 100 round trips, and then time 1,000 round trips over the same pair. One round trip consists of a `ping` message and its `pong` reply. Actor creation and termination are outside the timed interval.

Elapsed wall time was measured in microseconds. A short warm-up preceded the recorded native and Erlang series. The tables report medians: three observations for the browser lifecycle test, seven for the native lifecycle test, and twenty-one for Erlang; the native ping-pong median is from eleven observations and the Erlang median from twenty-one. Browser ping-pong observations required the additional precaution described in Section D.4.1.

The chained-spawning medians use seven native observations at 1,000 actors, five at 10,000 and 100,000, twenty-one Erlang observations at 1,000 and 10,000, and eleven at 100,000. The final isolated-node check uses nine observations at 1,000 actors.

The Web Prolog and Erlang programs use the same lifecycle and message protocols. In particular, links are disabled in Web Prolog where Erlang's ordinary unlinked spawning is used, and both versions wait for monitor notifications before considering an actor terminated. The complete sources are also provided as `benchmarks/actor_benchmarks.pl` and `benchmarks/actor_benchmark.erl` in the demonstrator repository.

D.3 Actor lifecycle

Web Prolog actors are linked by default, whereas Erlang's `spawn_monitor/3` monitors the new process without linking it. The benchmark therefore disables Web Prolog's default link explicitly. It measures the following complete actor lifecycle:

```
benchmark_lifecycle(Actors, Microseconds) :-
    get_time(T0),
    forall(between(1, Actors, _),
           benchmark_one_lifecycle),
    get_time(T1),
    Microseconds is round((T1-T0)*1_000_000).

benchmark_one_lifecycle :-
    spawn(benchmark_idle, Pid, [
        link(false),
        monitor(true),
        src_predicates([benchmark_idle/0])
    ]),
    Pid ! stop,
    receive({down(Pid, _, true) -> true}).

benchmark_idle :-
    receive({stop -> true}).
```

Only one child is live at a time. The test consequently measures the cost of repeatedly creating and reclaiming actors, not the maximum number of simultaneously live actors. Capacity and memory consumption require a separate experiment.

The corresponding Erlang operation is:

```
one_lifecycle() ->
    {Pid, Ref} = spawn_monitor(?MODULE, idle, []),
    Pid ! stop,
    receive
        {'DOWN', Ref, process, Pid, normal} -> ok
    end.
```

```
idle() ->
    receive stop -> ok end.
```

Table D.2 gives the median elapsed time for a sequence of complete lifecycles.

Table D.2 Median time for complete actor lifecycles, in milliseconds.

Actors	Browser node	Native node	Erlang
10	1 271	5.285	0.011
30	3 971	15.294	0.035
100	13 107	51.478	0.117

The near-linear progression is more informative than the largest number alone. In this implementation, one Worker actor lifecycle costs about 130 ms, one native actor lifecycle about 0.515 ms, and one BEAM process lifecycle about 1.2 microseconds. At 100 actors, the browser node is about 255 times slower than the native node. For this complete lifecycle operation, the full native node is about 440 times slower than Erlang.

These ratios describe the chosen end-to-end operation. The browser and native measurements include preparation of private Prolog execution contexts; the Erlang module is already loaded. The table should therefore be read as the price of the actor abstractions supplied by the three systems, not as a comparison of their lowest-level schedulers.

The native implementation does not pool actors or reuse their mutable modules. Every actor receives a fresh private context, but imports one immutable implementation of the standard I/O wrappers. Public sandboxed actors retain actor-local wrappers, since those predicates form part of the sandbox's distinction between actor I/O and general stream I/O.

D.3.1 Chained spawning

The lifecycle ratio must not be read as a raw spawning ratio. A different program, in which every actor creates the next actor in a chain, isolates a finer-grained creation pattern. With links made explicitly equivalent to Erlang's ordinary `spawn`, its Web Prolog form is:

```
start(Num) :-
    self(Self),
    start_proc(Num, Self).

start_proc(0, Pid) :- !,
    Pid ! ok.
```

```
start_proc(Num, Pid) :-
    Num1 is Num-1,
    spawn(start_proc(Num1, Pid), NPid, [
        link(false)
    ]),
    NPid ! ok,
    receive({ok -> true}).
```

Each child receives its `ok` message from its parent and the last actor sends an `ok` to the original caller. Unlike the lifecycle test, the program installs no monitor and does not serialise creation with a complete monitored shutdown.

The current stand-alone actor core runs the goal in the spawning module's context, without the private-module isolation added by the full node. Table D.3 compares this layer-0 core with Erlang.

Table D.3 Median time for the unlinked actor chain, in milliseconds.

Actors	Native actor core	Erlang	Ratio
1 000	30.926	0.589	52.5
10 000	307.601	6.242	49.3
100 000	3 081.924	58.060	53.1

The layer-0 spawning gap is therefore about 50 times. Running the same chain through the full native node still prepares a new private Prolog context at every generation, but imports the prepared actor-I/O implementation rather than compiling it again. It also omits the post-load repair pass when an actor has no source to load: without source, no imported predicate or hook-installed module state can have been shadowed.

The distribution layer allocates pids from an unpredictable ten-digit space. Its allocator draws five bytes from the process CSPRNG and rejects the short tail that would introduce modulo bias. This avoids per-thread pseudo-random initialisation while retaining unguessable identifiers.

The full native node completes the 1,000-actor chain in 74.170 ms, against 0.589 ms for Erlang – a ratio of about 126. Temporary module creation and destruction by themselves cost about 2.4 microseconds per actor on this machine; complete source-free module preparation raises that to about 8.8 microseconds. Fresh private contexts are therefore no longer the dominant cost in this test.

D.4 Steady-state message passing

Actor startup would dominate a short browser ping-pong test. The benchmark therefore starts its two actors first and reuses them:

```
benchmark_ping_once(Ping, N, Microseconds) :-
```

```
    self(Self),
    get_time(T0),
    Ping ! run(N, Self),
    receive({ping_done -> true}),
    get_time(T1),
    Microseconds is round((T1-T0)*1_000_000).

benchmark_ping_server(Pong) :-
    receive({
        run(N, From) ->
            benchmark_ping_rounds(N, Pong),
            From ! ping_done,
            benchmark_ping_server(Pong);
        stop ->
            true
    }).

benchmark_ping_rounds(0, _) :- !.
benchmark_ping_rounds(N, Pong) :-
    self(Self),
    Pong ! ping(Self),
    receive({pong -> true}),
    N1 is N-1,
    benchmark_ping_rounds(N1, Pong).

benchmark_pong_server :-
    receive({
        ping(Ping) ->
            Ping ! pong,
            benchmark_pong_server;
        stop ->
            true
    }).
```

The omitted setup routine starts and monitors both actors, with their default links disabled. It performs the warm-up and calls `benchmark_ping_once/3` for each observation. Finally, it stops both actors and receives both monitor notifications. The Erlang program uses the same sequence.

Moving actor creation outside the timed interval narrows the gap, but does not remove it. The native node completes about 65 times as many round trips per second as the browser node. Erlang in turn completes about 29 times as many as the native node. The browser path crosses two Worker boundaries and the page-level relay on every round trip; the native and BEAM paths remain inside one runtime.

Table D.4 Steady-state time and throughput for 1,000 round trips.

Substrate	Median time	Round trips/s	Relative rate
Browser node	1 110 ms	901	1.0
Native node	17.087 ms	58 524	65.0
Erlang	0.580 ms	1 724 138	1 914.0

D.4.1 A browser measurement caveat

Repeated browser batches in one page session did not behave as independent observations. Five consecutive batches of 100 round trips over one actor pair took 33, 49, 66, 84, and 103 ms. Recreating the pair did not remove all of the drift, whereas reloading the page did. The cause has not yet been isolated; possible locations include the Worker relay, retained mailbox or routing state, and demonstrator instrumentation.

The browser result in Table D.4 is therefore the median of three observations made in freshly loaded page sessions, followed by a fourth foreground run that reproduced the result. It should be treated as an indicative measurement of the current demonstrator, not as a stable limit of SWI-WASM or browser actors. The drift is also an actionable performance finding: before larger browser benchmarks are meaningful, the retained state responsible for it should be identified.

D.5 RPC, pagination, and query placement

RPC performance belongs to a separate transport and placement experiment. Consider, for example, a WordNet query distributed over two HTTP nodes:

```
rpc(URI1, type(S, v)),
rpc(URI2, word(S, W))
```

If `type(S,v)` produces many solutions, the second call is made once for each of them. The elapsed time then measures an N+1 remote-query plan: HTTP request construction, network round trips, connection reuse, and repeated remote setup. Options such as the solution-page `limit` and HTTP keep-alive policy can change the result substantially without changing the logical meaning of the query.

Moving a join to the node that holds the relevant data, batching requests, keeping connections alive, and choosing a suitable page size can therefore matter more than the raw speed of either Prolog engine. A reproducible RPC benchmark should distinguish local execution, one warm persistent connection, one fresh connection per call, and a genuinely remote network path; record the dataset and node policies; and report both latency to the first answer and throughput for all answers.

D.6 Interpretation and limits

For these two workloads, the ordering is unambiguous: the browser node is slowest, the native node is substantially faster, and Erlang is fastest by a wide margin. The reasons are architectural. A browser actor is a strong isolation boundary containing a Prolog virtual machine. A native actor is a lighter thread and Prolog context. A BEAM process is the unit for which the Erlang virtual machine has been specialised over several decades.

That ordering should not be generalised beyond the evidence:

- The results come from one machine and one set of runtime versions.
- The browser measurements use the demonstrator in an embedded Chromium-based browser, whose exact browser-engine build was not exposed.
- The benchmark messages are small local terms. It does not measure large-term copying, cross-node communication, HTTP, WebSocket transport, or application computation.
- Lifecycle measurements include source and context preparation. A different browser architecture – for example, several actors in one Worker – would answer a different isolation and performance question.
- No claim is made here about maximum actor population or memory per actor. Those require a capacity benchmark with resource measurements and well-defined failure criteria.

The comparison nevertheless gives useful engineering direction. Browser actors should be used at a granularity that justifies a Worker and an independent Prolog VM, rather than as replacements for millions of tiny BEAM processes. Native actors can be substantially finer-grained, although their private Prolog contexts still make them much heavier than Erlang processes. The public actor semantics remain common across the substrates; what differs is the cost at which each implementation currently provides them.